

UrbanPulse Lambda vs Kappa Matrix

UrbanPulse Lambda vs Kappa Matrix

| Dimension | Lambda | Kappa |

| :--- | :--- | :--- |

| Latency | UrbanPulse uses Flink for sub-minute incident alerts while Spark produces monthly and ward-level government reports. | UrbanPulse would need the same latency and reprocessing story in one engine, which is harder to operate. |

| Fault Tolerance | UrbanPulse keeps both real-time and historical paths so alerts and reports continue even if one layer is replayed. | UrbanPulse would have to replay the whole stream to recover, which slows incident handling. |

| Operational Complexity | UrbanPulse uses Kafka + Flink + Spark but keeps the architecture explicit for real-time and batch needs. | UrbanPulse would simplify the stack but weaken the separation between real-time alerts and historical reporting. |

| Reprocessing | UrbanPulse can replay historical messages into Spark for monthly government reporting without interrupting real-time alerts. | UrbanPulse would need to reprocess the same data into a single logical pipeline, increasing operational risk. |

| Cost | UrbanPulse uses Kafka for durable retention and Spark/Flink for targeted processing, which is justified by the mixed workload. | UrbanPulse would likely need more expensive replay and recovery logic for the same output quality. |

| Government Reporting | UrbanPulse uses Spark and Parquet outputs to satisfy public-sector reporting and audit requirements. | UrbanPulse can produce reports but would struggle to combine a low-latency alerting path with long-term compliance reporting. |

Architecture Choice

Lambda Architecture is the correct choice for UrbanPulse because real-time alerts and monthly government reports must coexist.